

2.17 Application Flow Control with Navigation Frameworks

Lesson Objectives

By the end of this lesson, you will be able to:

1. **Explain** why dedicated navigation libraries are needed in React Native and how React Navigation compares to React Router in web apps
2. **Implement** tab, drawer, and stack navigation using React Navigation, registering screens with `NavigationContainer` and the appropriate navigator
3. **Pass** data between screens using route parameters and access the navigation API from any component using the `useNavigation` hook

Recap - What We Covered in 2.15 and 2.16

Lesson	Topics
2.15	<code>ScrollView</code> , <code>SafeAreaView</code> , <code>KeyboardAvoidingView</code> , flex layout, <code>Pressable</code>
2.16	A complete single-screen app: <code>FlatList</code> , component extraction, color tokens

Every app so far has had exactly one screen. Today that changes.

Navigation in Mobile Apps

- A real mobile app almost always has **multiple screens**: a list, a detail view, a settings page
- On the web, the browser provides navigation for free: URLs, browser history, the Back button
- **Mobile apps have no browser and no URL bar.** There is no built-in history stack to move between screens
- React Native needs a dedicated library to fill this gap: **React Navigation**

React Navigation vs React Router

Recall from Lesson 2.8: React Router manages **client-side routing**, matching a URL to a component, on top of the browser's History API.

React Navigation solves the same underlying problem, moving between screens while keeping track of history, but for an environment with no URL and no browser:

	React Router (web)	React Navigation (native)
Backed by	Browser History API	Its own in-memory navigation state
Addressing	URL paths (/customers/:id)	Screen names + route params
Building blocks	Route , Link , useNavigate	Navigator , Screen , navigation.navigate
Back button	Browser or <a> / Link	Provided by the navigator (stack)

Same problem, same declarative shape, different platform underneath.

The Problem: Navigating Without a Library

Before installing anything, try switching screens with just `useState` :

```
// App.js
const [currentScreen, setCurrentScreen] = useState("Home");

let content;
if (currentScreen === "Home") content = <HomeScreen />;
else if (currentScreen === "Settings") content = <SettingsScreen />;

return (
  <View>
    <Button title="Home" onPress={() => setCurrentScreen("Home")} />
    <Button title="Settings" onPress={() => setCurrentScreen("Settings")} />
    {content}
  </View>
);
```

Works, but: no "back" behaviour, and every new screen means another `if/else` branch and another button.

React Navigation

React Navigation is a **standalone library**, not part of React Native itself.

It provides three main navigator types, each suited to a different navigation pattern:

- **Bottom Tabs** - a fixed row of tabs for a handful of top-level, equally-important sections
- **Drawer** - a slide-in side panel, for when there are more sections than comfortably fit in a tab bar
- **Stack** - a history-based navigator that pushes and pops screens, for drill-down flows

All three share the same underlying package and configuration pattern.

Most navigators ship in two flavours: a **JS-based** version (fully customisable, rendered in JavaScript) and a **native** version (backed by the platform's own navigation components, generally smoother and more performant). This lesson uses **native stack** throughout, the recommended default for Stack in the React Navigation version installed here. A **native bottom tabs** navigator also exists and is on track to become the library's default in an upcoming release, but in the version this lesson installs it is still opt-in and its docs warn the API may change in a minor release, so this lesson uses the standard, stable Bottom Tabs navigator instead.

Installing React Navigation

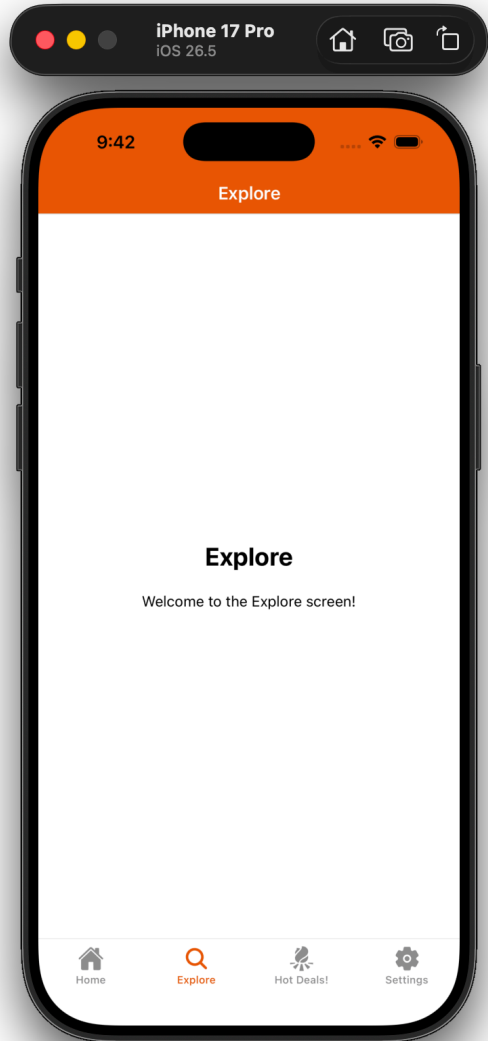
Install the core package and its required dependencies once. They are shared across every navigator used in this lesson:

```
npm install @react-navigation/native  
npx expo install react-native-screens react-native-safe-area-context
```

- `@react-navigation/native` provides `NavigationContainer`, the top-level wrapper every navigator needs
- `react-native-screens` lets React Navigation use each platform's native view management for better performance
- `react-native-safe-area-context` lets screens render content inside the safe area automatically

A specific navigator package (tabs, drawer, or stack) is installed separately, as each is introduced.

Part 2: Tab Navigation



A fixed row of tabs, usually pinned to the bottom of the screen, each tab switching to a different top-level section.

- Every tab is a **peer**: none is "more important" or nested under another
- The active tab is always visible, so users always know where they are and how to reach any other section in one tap
- **Use it when**: the app has a small number (roughly 3-5) of equally important, frequently visited sections
- **Examples**: Instagram's Home / Search / Reels / Profile bar, most shopping apps (Home, Categories, Cart, Account)

Building a Tab Navigator

```
npm install @react-navigation/bottom-tabs
```

```
import { NavigationContainer } from "@react-navigation/native";
import { createBottomTabNavigator } from "@react-navigation/bottom-tabs";

const Tab = createBottomTabNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator>
        <Tab.Screen name="Home" component={HomeScreen} />
        <Tab.Screen name="Settings" component={SettingsScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

Anatomy of a Navigator

- `NavigationContainer` is the top-level wrapper that manages the navigation tree and holds navigation state. Every React Navigation app needs it at the root
- `createBottomTabNavigator()` returns an object (`Tab`) with two components: `Tab.Navigator` and `Tab.Screen`
- `name` is a unique identifier used for navigation, and becomes the header title and tab label by default

Common mistake: `component={<HomeScreen />}` passes a rendered element, not a component reference. This causes React Navigation to create a new component type on every render, losing screen state. Always pass `component={HomeScreen}` .

The `options` Prop

Every `Screen` component accepts an `options` prop: an object (or a function returning one) that customises how that one screen is presented, without touching its component code.

- **Header:** `headerTitle`, `headerShown`, `headerStyle`, `headerTintColor`
- **Tab bar** (Bottom Tabs only): `tabBarLabel`, `tabBarIcon`, `tabBarBadge`
- **Drawer** (Drawer only): `drawerLabel`, `drawerIcon`

Today's lab uses a handful of these. The same `options` prop exists on `Tab.Screen`, `Drawer.Screen`, and `Stack.Screen` alike, only which properties apply differs by navigator.

Customising Titles and Labels

The most common use: the screen identifier does not read well as a title, so `headerTitle` and `tabBarLabel` override it:

```
<Tab.Screen
  name="HotDeals"
  component={HotDealsScreen}
  options={{
    headerTitle: "🔥 Hot Deals!",
    tabBarLabel: "Hot Deals!",
  }}
/>
```

- `headerTitle` - text in the top header bar
- `tabBarLabel` - label under the tab icon
- `initialRouteName` on `Tab.Navigator` changes which screen shows on launch, without reordering tabs

Styling Every Screen at Once

`screenOptions` on `Tab.Navigator` applies styling to every registered screen:

```
<Tab.Navigator
  initialRouteName="Explore"
  screenOptions={{
    headerStyle: { backgroundColor: Colors.PRIMARY },
    headerTintColor: Colors.WHITE,
    tabBarActiveTintColor: Colors.PRIMARY,
  }}
>
```

- `headerStyle` - background colour of the header
- `headerTintColor` - colour of the header title text
- `tabBarActiveTintColor` - colour of the active tab icon and label

The same properties set here also work inside a single `Tab.Screen`'s `options`, for a one-off override.

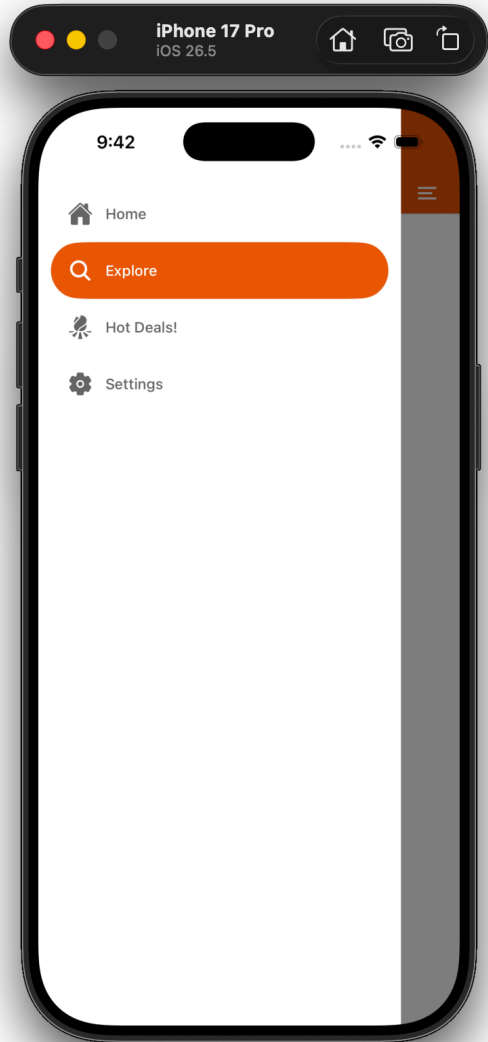
Tab Icons

```
import Ionicons from "@react-native-vector-icons/ionicons";

<Tab.Screen
  name="Home"
  component={HomeScreen}
  options={{
    tabBarIcon: ({ color, size }) => (
      <Ionicons name="home" size={size} color={color} />
    ),
  }}
/>;
```

- `tabBarIcon` takes a function, called on every tab bar render
- `color` and `size` are supplied automatically to match the active/inactive state
- Icon packs are installed individually, for example `@react-native-vector-icons/ionicons`. Browse names at icons.expo.fyi

Part 3: Drawer Navigation



A slide-in side panel, opened with a swipe gesture or a menu icon, listing every top-level section as a menu item.

- Scales to **many more sections** than a tab bar can comfortably show, since the panel scrolls if needed
- Hidden by default, so it costs no permanent screen space, at the cost of being less discoverable than a visible tab bar
- **Use it when:** there are more top-level sections than fit in a tab bar, or when sections are used less frequently
- **Examples:** Gmail's folder/label menu, most admin dashboards

Building a Drawer Navigator

The configuration pattern is nearly identical to tabs.

```
npx expo install react-native-gesture-handler react-native-reanimated react-native-worklets  
npm install @react-navigation/drawer
```

These three are general-purpose libraries used well beyond the drawer, e.g. swipeable lists, pinch-to-zoom, bottom sheets:

- `react-native-gesture-handler` - recognises touch gestures (pan, swipe, pinch, rotation) natively, on the UI thread instead of the JS thread, so gestures stay responsive even when JS is busy. The drawer uses it for the open/close swipe
- `react-native-reanimated` - runs animation logic on the UI thread instead of the JS thread, for smooth 60fps animations. The drawer uses it for the slide-in transition
- `react-native-worklets` - runs small JS functions ("worklets") directly on the UI thread. Reanimated 4 depends on it to move that animation logic off the JS thread

Using the Drawer Navigator

```
import { createDrawerNavigator } from "@react-navigation/drawer";

const Drawer = createDrawerNavigator();

function DrawerNavigator() {
  return (
    <Drawer.Navigator>
      <Drawer.Screen name="Home" component={HomeScreen} />
      <Drawer.Screen name="Explore" component={ExploreScreen} />
      <Drawer.Screen name="HotDeals" component={HotDealsScreen} />
      <Drawer.Screen name="Settings" component={SettingsScreen} />
    </Drawer.Navigator>
  );
}
```

Same shape as the tab navigator: a `Navigator` wrapping `Screen` components, each with `name` and `component`.

Drawer Options and Styling

Per-screen (`Drawer.Screen`)

```
options={{  
  headerTitle: "🔥 Hot Deals!",  
  drawerLabel: "Hot Deals!",  
}}
```

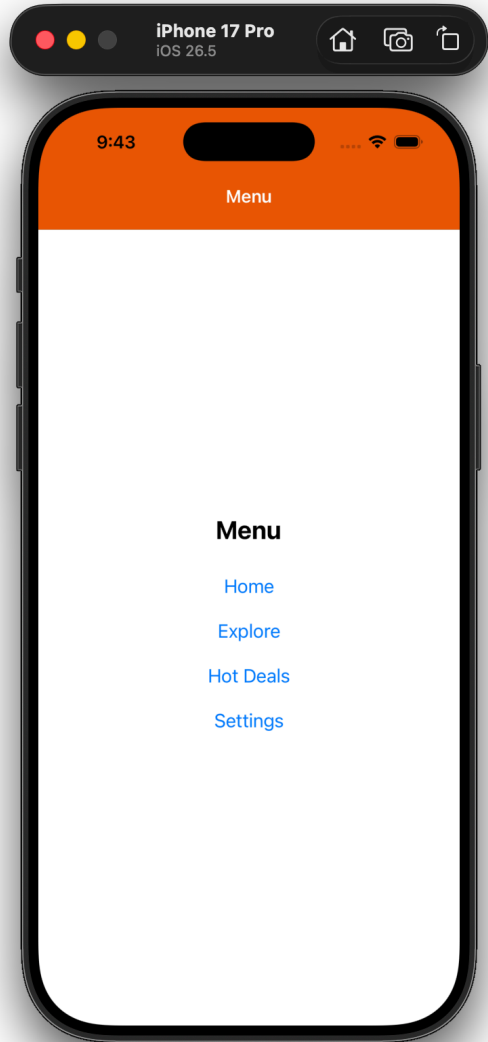
`drawerLabel` is the drawer equivalent of `tabBarLabel`.

`drawerIcon` works exactly like `tabBarIcon`, receiving `{ color, size }`.

All screens (`Drawer.Navigator`)

```
screenOptions={{  
  headerStyle: { backgroundColor: Colors.PRIMARY },  
  headerTintColor: Colors.WHITE,  
  drawerActiveBackgroundColor: Colors.PRIMARY,  
  drawerActiveTintColor: Colors.WHITE,  
}}
```

Part 4: Stack Navigation



Manages a **history** of screens, not a flat list of peers: navigating pushes a new screen on top, and pressing back pops it off.

- Screens are **hierarchical**, not equal: a detail screen only makes sense after arriving from a list
- Provides a back button and swipe-back gesture automatically, since it always knows the previous screen
- **Use it when:** the flow is a drill-down, list to detail, form wizard, or checkout steps
- **Examples:** opening a product from a search results list, a multi-step signup form

Building a Stack Navigator

```
npm install @react-navigation/native-stack
```

Two stack packages exist:

- `@react-navigation/stack` - pure JavaScript, fully customisable
- `@react-navigation/native-stack` - uses native platform navigation, more performant

Use `native-stack` unless there is a specific reason not to.

Building a Stack Navigator

```
import { createNativeStackNavigator } from "@react-navigation/native-stack";

const Stack = createNativeStackNavigator();

function StackNavigator() {
  return (
    <Stack.Navigator
      screenOptions={{
        headerStyle: { backgroundColor: Colors.PRIMARY },
        headerTintColor: Colors.WHITE,
      }}
    >
      <Stack.Screen name="Menu" component={MenuScreen} />
      <Stack.Screen name="Home" component={HomeScreen} />
      <Stack.Screen name="HotDeals" component={HotDealsScreen} />
    </Stack.Navigator>
  );
}
```

Unlike Drawer or Tabs, Stack has no built-in menu UI, hence the dedicated `MenuScreen` as an entry point.

The `navigation` Prop

React Navigation automatically provides a `navigation` prop to every **registered screen** component:

```
function MenuScreen({ navigation }) {  
  return <Button title="Home" onPress={() => navigation.navigate("Home")} />;  
}
```

- `navigation.navigate("ScreenName")` moves to a screen by name
- Also available on Tab and Drawer screens, not just Stack
- The header back arrow appears automatically whenever there is a previous screen to return to

Limitation: the prop is only available to components registered directly as screens.

The `useNavigation` Hook

Moving the buttons into a child component (e.g. `MenuOptions`) means `navigation` is no longer passed down as a prop. Passing it manually through every layer is awkward.

```
import { useNavigation } from "@react-navigation/native";

function MenuOptions() {
  const navigation = useNavigation();

  return <Button title="Home" onPress={() => navigation.navigate("Home")} />;
}
```

`useNavigation` reads the navigation object from context, so it works in **any** component in the tree, not only registered screens.

Passing Data with Route Parameters

`navigate` accepts a second argument: an object of route params passed to the destination screen.

```
navigation.navigate("ProductDetail", {  
  product: "Apple iPad",  
  id: 123,  
  price: 299,  
});
```

```
// ProductDetailScreen.js  
import { useRoute } from "@react-navigation/native";  
  
function ProductDetailScreen() {  
  const { params } = useRoute();  
  return (  
    <Text>  
      {params.product} - ${params.price}  
    </Text>  
  );  
}
```

`useRoute` is the route-object counterpart to `useNavigation` : available anywhere, not just on registered screens.

Dynamic Header Titles from Params

A static `options` object cannot reflect data known only at navigation time. Pass a **function** instead:

```
<Stack.Screen
  name="ProductDetail"
  component={ProductDetailScreen}
  options={({ route }) => ({
    title: `Product Detail: ${route.params.product}`,
  })}
/>
```

The callback receives `{ route }`, containing the params passed during navigation.

When the title depends on fetched data instead of params, use `navigation.setOptions` inside `useLayoutEffect`, which runs before paint and avoids a visible title flash. For params known at navigation time, the approach above is simpler.

Part 5: State Persistence Across Navigators

Whether a screen's local state survives navigating away and back depends on which navigator hosts it. This is a common source of confusion.

Consider a `HomeScreen` with a search `TextInput` backed by local `useState`, holding whatever the user has typed:

- In a **Stack** navigator, going back and returning to `HomeScreen`: the typed text is **gone**
- In a **Tab** or **Drawer** navigator, switching away and back: the typed text **remains**

Why State Behaves Differently

- **Stack:** navigating forward pushes a screen and mounts it; going back **pops and unmounts** it, destroying its state. The previous screen was never unmounted, so its own state is untouched
- **Tab / Drawer:** there is no stack of screens to pop. All registered screens **stay mounted** once visited; only visibility toggles. Because the component instance is never destroyed, its state survives

Trade-off: Stack screens lose unsaved input or scroll position on back. Tab/Drawer screens avoid this, at the cost of keeping more screens in memory simultaneously. Choose the navigator with this in mind.

Resetting State with `useFocusEffect`

To clear state every time a screen gains focus, **regardless of navigator type**:

```
import { useFocusEffect } from "@react-navigation/native";
import { useCallback, useState } from "react";

function HomeScreen() {
  const [searchQuery, setSearchQuery] = useState("");

  useFocusEffect(
    useCallback(() => {
      setSearchQuery("");
    }, []),
  );
}
```

The callback **must** be wrapped in `useCallback` with an empty dependency array, otherwise a new function reference is created every render, and the effect fires on every render instead of only on focus.

Comparing the Three Navigators

	Stack	Drawer	Bottom Tabs
Purpose	Manage navigation as a stack of screens	Side menu for switching sections	Bottom bar for switching sections
UI	Card-style slide in/out	Panel slides in from the side	Tabs with icons/labels
State	Destroyed on pop; kept while pushed	Persists, screens stay mounted	Persists, screens stay mounted
Pros	Flexible, well suited to deep drill-down flows	Scales to many routes, a familiar "hamburger menu" pattern	Intuitive for mobile users, supports icons and badges, quick switching
Cons	Deep nesting can become hard to follow	Hidden by default, so options are less discoverable	Usable range is roughly 3-5 tabs before the bar gets cramped

Part 6: Nesting Navigators

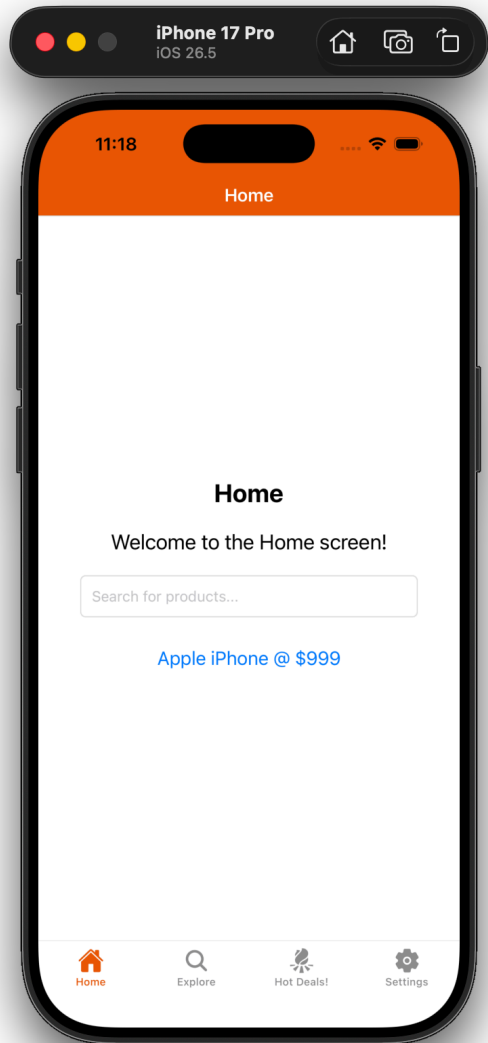
A single navigator cannot satisfy every requirement at once:

- **Bottom Tabs alone:** no way to push a detail screen (e.g. `ProductDetailScreen`) on top of the current tab with a back button, tabs are peers, not a history. Registering it as a `Tab.Screen` anyway would also add it as an unwanted extra tab, every screen registered on a Tab or Drawer navigator becomes a menu item
- **Stack alone:** no persistent tab bar for quick switching between top-level sections

Real apps usually need both **at the same time**: a persistent tab bar for top-level sections, plus other screens (detail views, forms) that are reachable by navigating to them but should never appear as menu items themselves.

Nesting solves this: place one navigator inside another, so each manages its own screens.

How Nesting Works



Register a navigator component itself as a `Stack.Screen`, alongside ordinary screens:

```
<Stack.Navigator>
  <Stack.Screen name="BottomTabs" component={BottomTabsNavigator} />
  <Stack.Screen name="ProductDetail" component={ProductDetailScreen} />
</Stack.Navigator>
```

- `BottomTabsNavigator` renders as one "screen" from the outer stack's point of view, tabs and all
- `ProductDetail` is registered on the **outer** stack, not inside the tab navigator, so it is navigable from any tab without ever becoming a tab itself
- Because it is a sibling of `BottomTabs` in the same outer stack, navigating to it from inside any tab pushes it on top of the tab bar, and the back button pops back to whichever tab it came from

Part 7: Authenticated Navigation

(Only if time permits - requires the Context API from Lesson 2.6)

The standard React Navigation pattern for login flows: maintain **two separate navigator stacks**, and conditionally render one based on authentication state.

```
function NavigationApp() {  
  const { isAuthenticated } = useContext(AuthContext);  
  return isAuthenticated ? <AppStackNavigator /> : <AuthStackNavigator />;  
}
```

Swapping the entire navigator on login/logout unmounts the old stack and mounts the new one, so there is no back button into the previous stack and no need to manually clear navigation history.

Common Pitfalls

- **Passing JSX instead of a component reference:** `component={<HomeScreen />}` forces remounts and loses state. Use `component={HomeScreen}`
- **Forgetting `useCallback` with `useFocusEffect`** : without it, the effect runs on every render, not just on focus
- **Expecting `navigation` in a non-screen component:** it is only auto-injected into registered screens. Use `useNavigation` (or `useRoute`) elsewhere
- **Installing `@react-navigation/drawer` before `react-native-reanimated`** : can force an incompatible Reanimated version and cause an `ERESOLVE` error

Workshop

Open `lesson.md` to begin the hands-on lab.

Q&A

End of Lesson - Exit Ticket